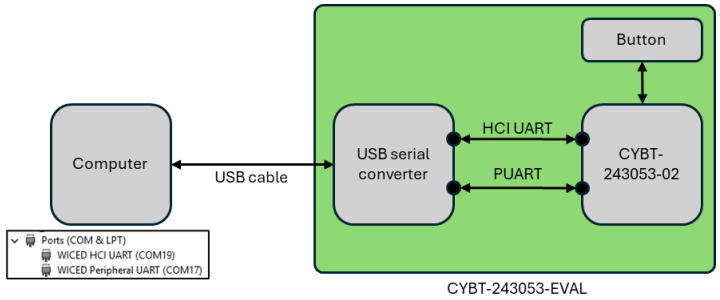
Dev kit ..... 2
Example use case ..... 3
Tools ..... 4
ClientControl ..... 4
TeraTerm ..... 5
Serial Bluetooth Terminal ..... 6
Modus Toolbox ..... 6
HCI mode / Application mode ..... 7
Project files ..... 8
Information about API ..... 9
Connect the CYBT-243053-EVAL to your device ..... 10
BT SPP Test ..... 12
Using Android ..... 12
Using Windows ..... 14
BLE HID Test ..... 15
Troubleshooting: no device found during scanning ..... 16
NVRAM usage ..... 17
Send report (BLE HID) ..... 18
Report descriptor ..... 20
Connected or not ..... 21
Update battery level ..... 21
Useful links ..... 22

# Dev kit




**COM19: HCI UART**

**Remark: might be another number on your computer.**

Used for 2 purposes:

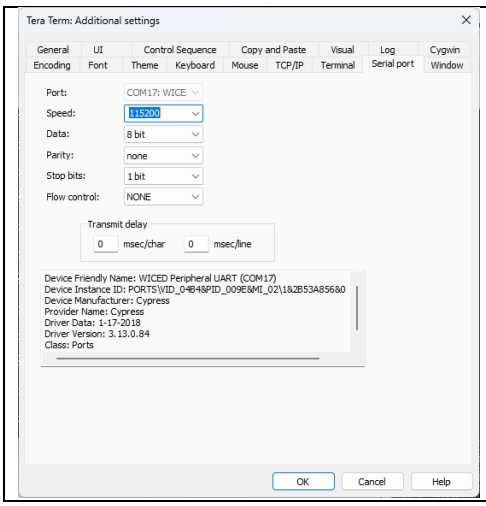
- Programming of the firmware
- Control over HCI (use the Infineon's tool ClientControl.exe to test it)

**COM17: Peripheral UART**

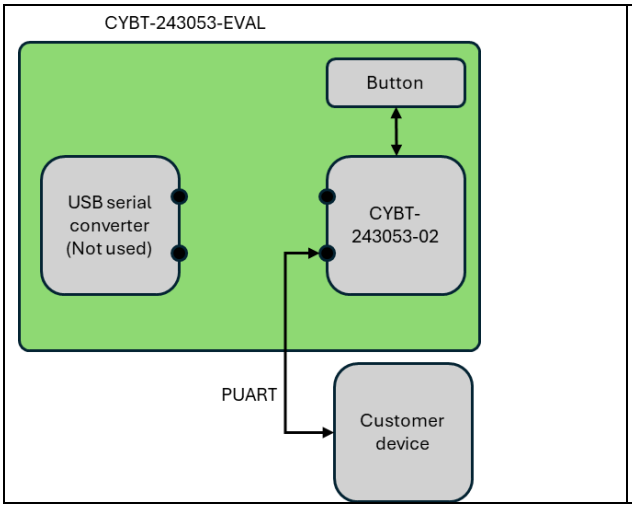
**Remark: might be another number on your computer.**

Used for 2 purposes:

- See the logs (if you called `wiced_set_debug_uart(WICED_ROUTE_DEBUG_TO_PUART)` )
- Emulate device: writing to it will be catch by the firmware (*`puart_rx_interrupt_callback`*)

| | |
|---|---|
|   | Configuration to be used. |

# Example use case




- Button can be used to enable pairing / advertisement
- Customer device send data packet to be transmitted over PUART.
    - If BT SPP connected, packet transmitted over BT SPP
    - If BLE HID connected, packet transmitted over BLE HID

The Windows computer decides which type of communication it wants to use (see BT SPP Test and BLE HID Test sections).

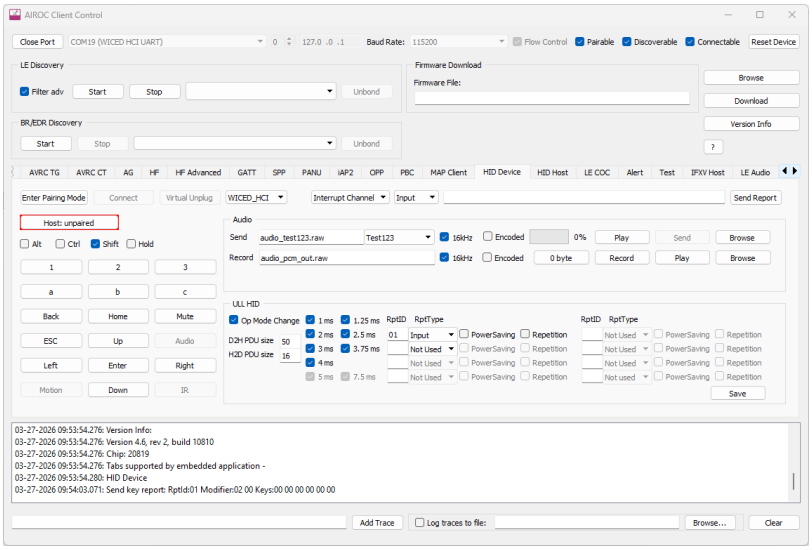
In case of BT SPP: transparent COM port.

In case of BLE HID: the CYBT-243053-EVAL “emulates” a keyboard.

# Tools

## ClientControl

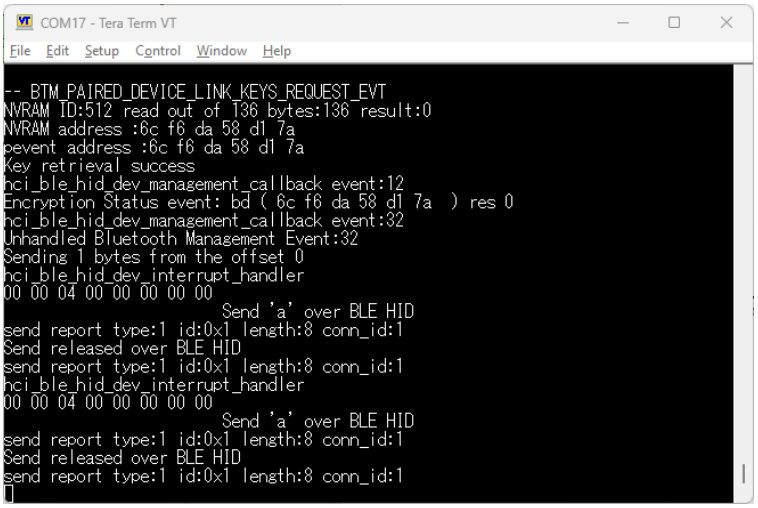
https://github.com/Infineon/btsdk-host-apps-bt-ble




To be used to debug using HCI interface.

# TeraTerm

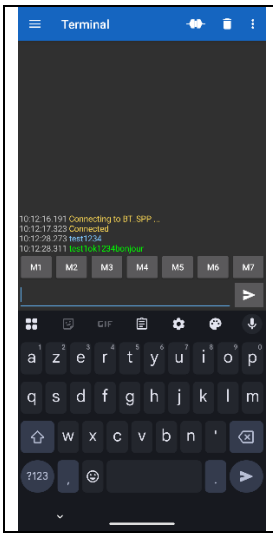
https://teratermproject.github.io/index-en.html



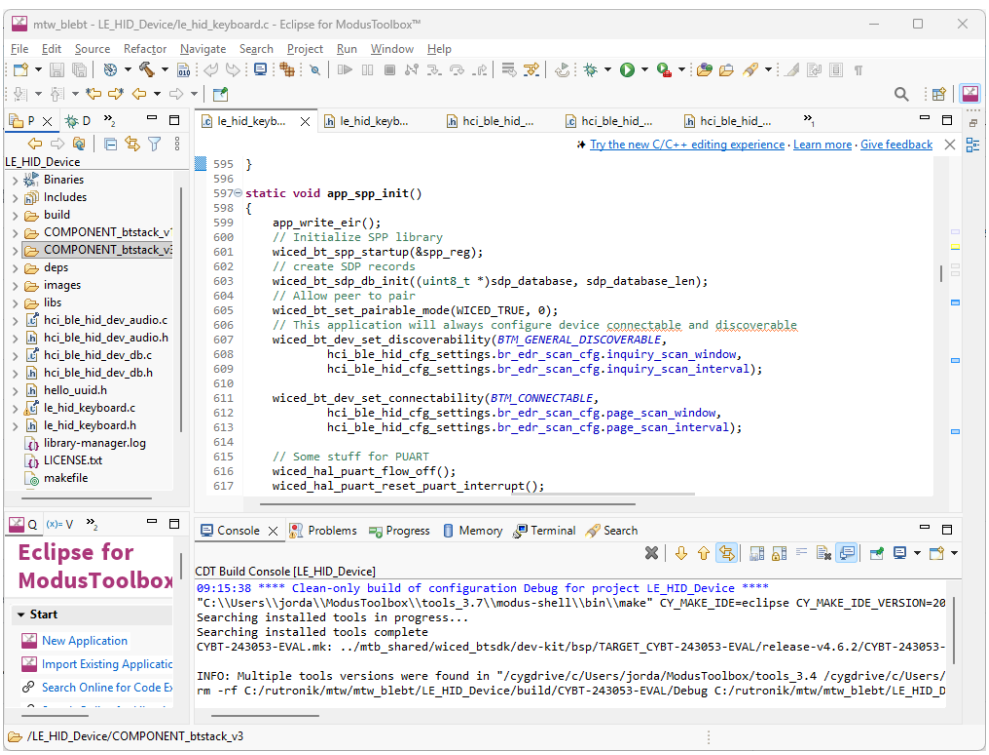

To be used to debug what happens.

## Serial Bluetooth Terminal

https://play.google.com/store/apps/details?id=de.kai_morich.serial_bluetooth_terminal&pcampaignid=web_share

| | |
|---|---|
|  | Use it to debug the BT SPP communication.<br><br>If Peripheral UART is opened on the laptop, writing to the port will be send over BT SPP to the app and will be displayed. |

## Modus Toolbox




Used to develop the firmware and program the device.

# HCI mode / Application mode

From https://infineon.github.io/btsdk-docs/BT-SDK/AIROC-HCI-Control-Protocol.pdf

A real Bluetooth® product could have an onboard MCU that uses CYWxxxxx device to provide Bluetooth® functionality. For such a product, MCU software would likely be used to control the device through a UART or SPI interface via a protocol that allows the MCU to send and receive commands, events, and data. This document describes a sample protocol for communication between an MCU and a CYWxxxxx device.

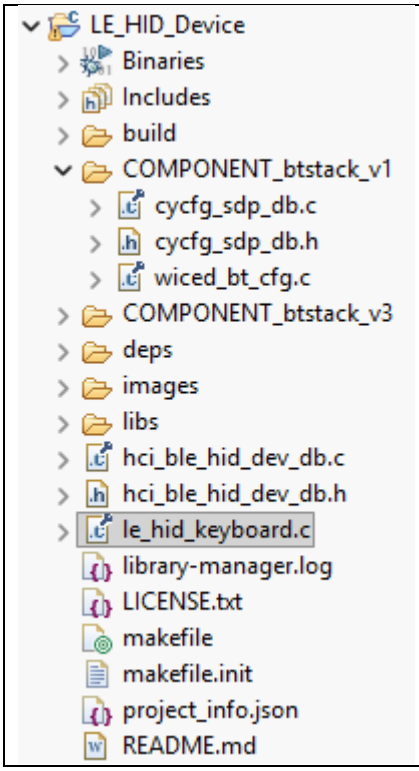
The CYWxxxxx devices support two operating modes: **the Bluetooth® Host Controller Interface (HCI) mode** and the **application mode**.

In the Bluetooth® HCI mode, the embedded stack in the device is not exercised and the device behaves as a standard Bluetooth® HCI controller. A standard Bluetooth® HCI controller supports the Bluetooth® HCI interface as defined in the Bluetooth® core specification.

In the Application mode, the embedded stack in the CYWxxxxx device is used and the device does not behave as a standard Bluetooth® controller.

In this example, the device runs in “application mode” -> we are not using the HCI interface, but the WICED BT SDK that enables to use BT and BLE.

# Project files




cycfg_sdp_db.c -> Configuration of the BT SPP profile.

wiced_bt_cfg.c -> Configuration of the BT/BLE module.

hci_ble_hid_dev_db.c -> Configuration of the BLE Gatt database.

le_hid_keyboard.c -> Logic implementation.

# Information about API

```
wiced_bt_stack_init( &hci_ble_hid_dev_management_callback,
&hci_ble_hid_cfg_settings , hci_ble_hid_cfg_buf_pools);
```

Initialize the BT/BLE stack. First parameter is a “hook” function that will be called by the BT/BLE stack in case of event (like the stack is ON, a connection is requested, ....).

```
hci_ble_hid_dev_management_callback
```

This function needs to handle all the events coming from the BT/BLE stack (big switch/case structure).

```
hci_ble_hid_dev_interrupt_handler
```

This function is called when the use press the button. You can use it for example to start advertisement of the BLE again if needed.

This function is used as test at the moment, and will send something in case connected over BT SPP or BLE HID.

```
spp_rx_data_callback
```

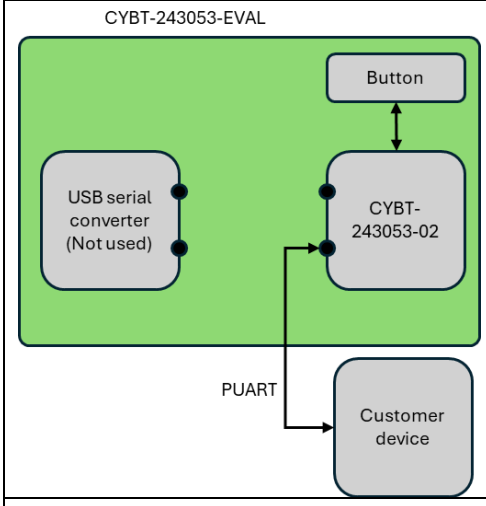
This function is called when the computer sends data to the BT/BLE module over BT SPP.

```
puart_rx_interrupt_callback
```

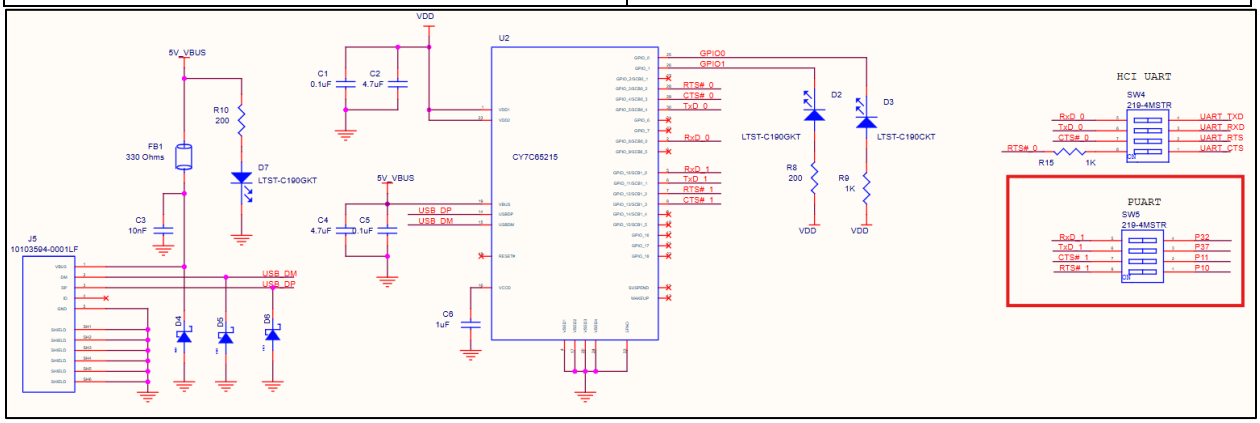
This function is called when the BT/BLE module receives something over the PUART interface.

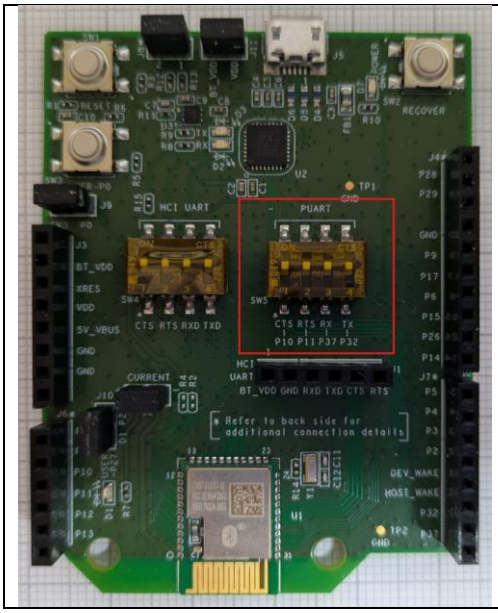
Use it to send information to the computer over BT SPP or BLE HID.

# Connect the CYBT-243053-EVAL to your device




PUART will be connected to your device and not to a debug computer.

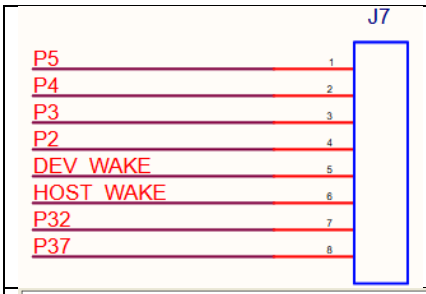
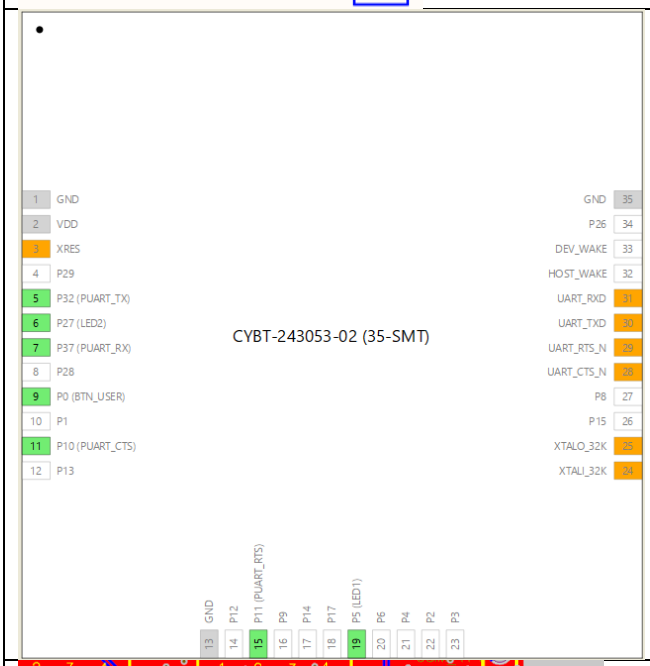
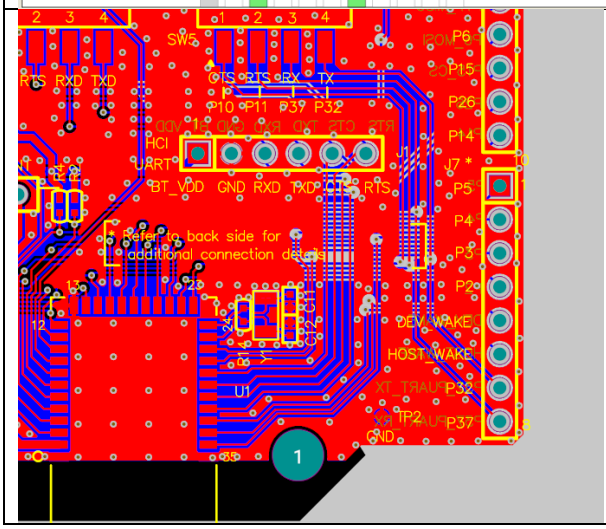



The PUART switches needed to be changed.

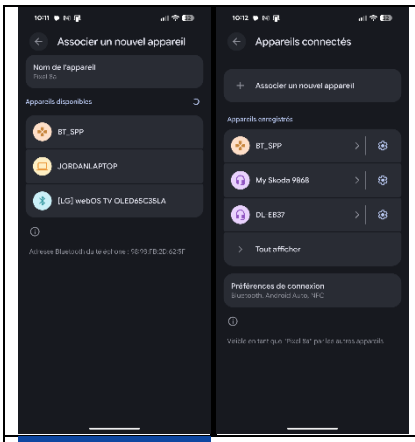
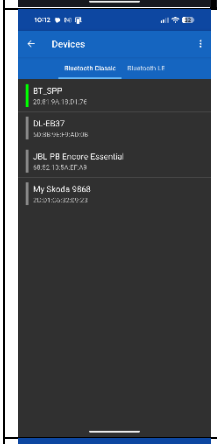
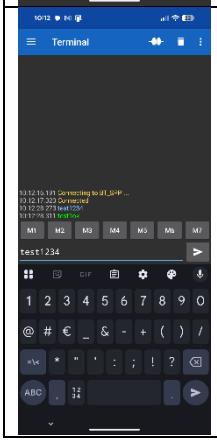
In the position of the picture, PUART is connected to the USB/Serial converter.

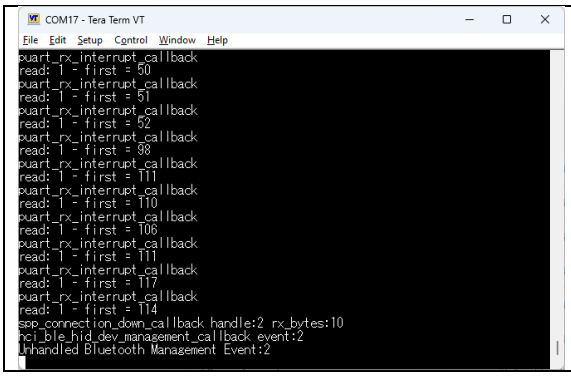
Set them to the other position (P10, P11, P37 and P32) to disconnected it from the USB/Serial converter and use the Arduino connector.

| | |
|---|---|
|  | RX connected to P37 and TX connected to P32. |
|  | P32 -> PUART_TX (output of the evaluation kit).<br><br>P37 -> PUART_RX (input of the evaluation kit). |
|  | P32 and P37 are available on the J7 connector. |

# BT SPP Test

## Using Android

| | |
|---|---|
|  | First need to pair the device with your Smartphone using the Bluetooth configuration. |
|  | Then, open the Serial Bluetooth Terminal app, go to Devices and select “BT_SPP”. This will open the connection with the BT SPP device. |
|  | You can write something in the app and send it using the “>>” button.<br><br>This will be catch inside the function “`spp_rx_data_callback`”. For debug purpose, the firmware send back something.<br><br>If you press the button of the dev kit, a message will be send from the device to the smartphone app. |

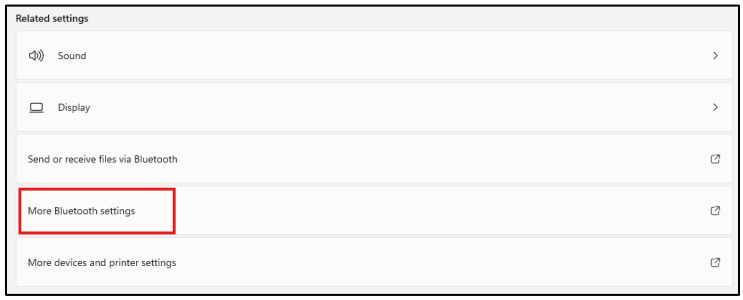



If you open the “Peripheral UART” COM port on your computer (115200 / no hardware flow control), and enter text using your keyboard, it will be catch by the function “`puart_rx_interrupt_callback`”.

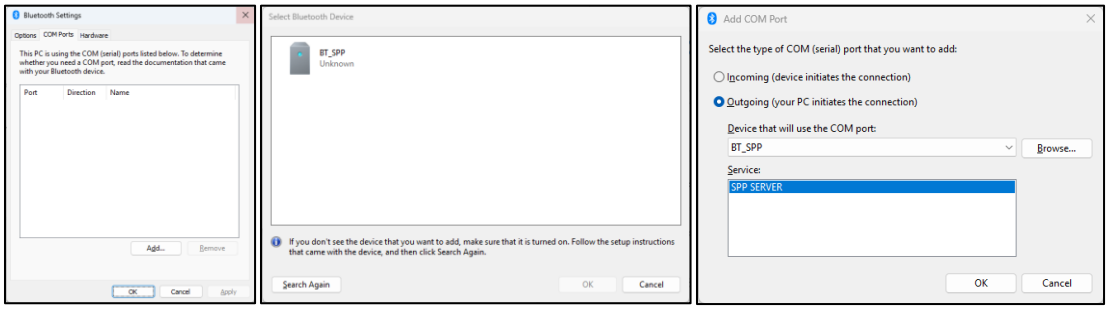
If connected over BT SPP, the message will be forwarded: `wiced_bt_spp_send_session_data(…)`

## Using Windows

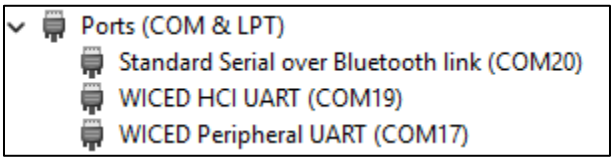
In the Bluetooth settings of Windows, press “More Bluetooth settings”:



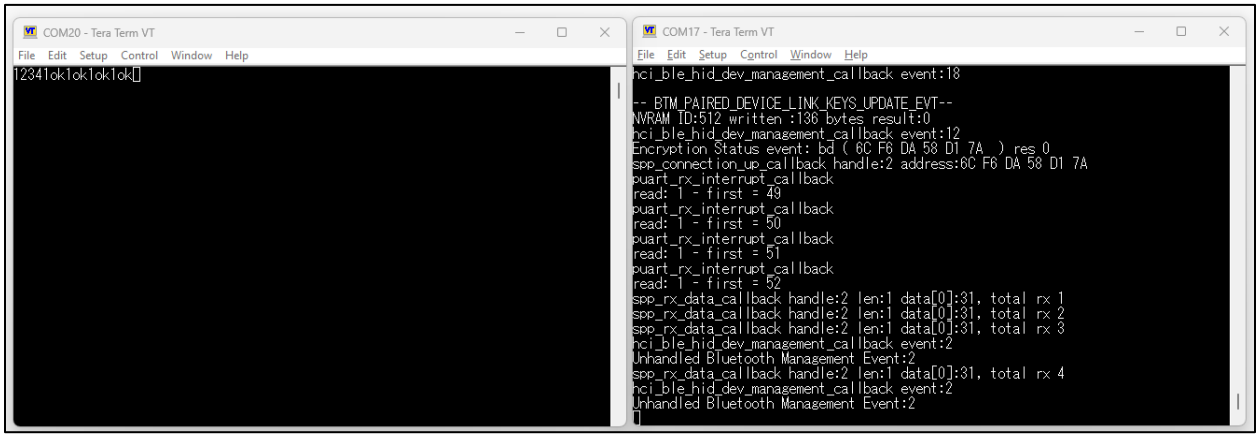

Then go to “COM Ports” and add a new one (select “Outgoing):




The new COM port should now appear in the device manager:

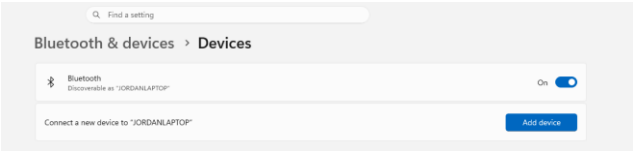



Open the COM20 (in this example), and you will be able to communicate:

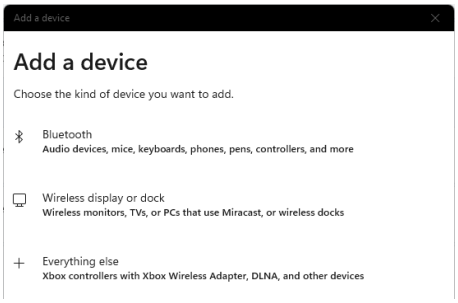


# BLE HID Test

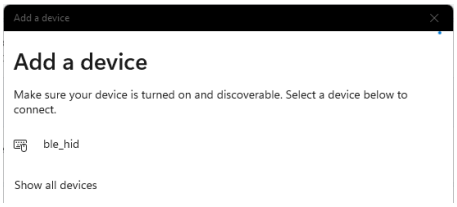
In the settings of Windows, you can add new Bluetooth device:



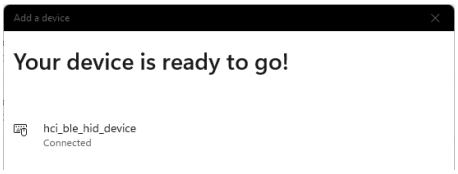

This will open such a window:



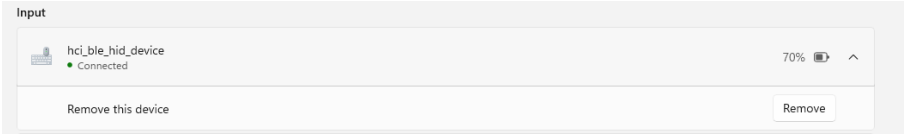

Click on “Bluetooth”, and the scan will start, and the available devices will be displayed:



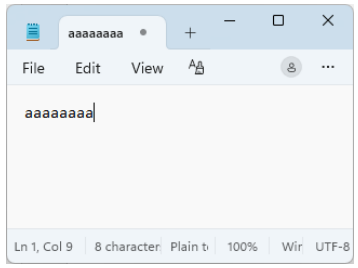

Click on “ble_hid” to pair the device:




After successful pairing, you will see the device in the list:



To test, simply open notepad, and press the USER_P0 button of the board. You should see:




Each time the button is pressed, a report is sent. See in the function "`hci_ble_hid_dev_interrupt_handler`":

```
if (hci_ble_hid_dev_state.conn_id != 0)
{
    // Size: HCI_BLE_HID_DEV_INPUT_REPORT_SIZE = 8
    uint8_t record_data_a[] = {0, 0, 4, 0, 0, 0, 0, 0}; // -> 'a'
    uint8_t record_data_release[] = {0, 0, 0, 0, 0, 0, 0, 0}; // -> key released
    wiced_trace_array(record_data_a, sizeof(record_data_a));

    WICED_BT_TRACE("Send 'a' over BLE HID\r\n");
    hci_ble_hid_dev_handle_send_report(HID_REPORT_TYPE_INPUT, HID_INPUT_REPORT_ID, record_data_a, ( uint16_t ) sizeof(record_data_a));

    WICED_BT_TRACE("Send released over BLE HID\r\n");
    hci_ble_hid_dev_handle_send_report(HID_REPORT_TYPE_INPUT, HID_INPUT_REPORT_ID, record_data_release, ( uint16_t ) sizeof(record_data_release));
}
```

## Troubleshooting: no device found during scanning

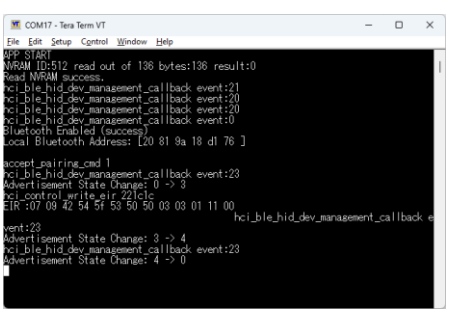
It might happen, that the device cannot be found, because the advertising is stopped after a while without connection.

At start, the device starts with "fast advertising" (Advertisement State Change: 0 -> 3).

After some time, the device goes into "slow advertising" (Advertisement State Change: 3 -> 4).

Then it stops advertising (Advertisement State Change: 4 -> 0).

In that case, the device cannot be found anymore.




**Remark: it is possible to start the advertising when pressing button for example.**

# NVRAM usage

Used to store the keys exchanged during the pairing.

See file:

```
mtb_shared/wiced_btsdk/dev-kit/baselib/20820A1/release-v4.6.2/COMPONENT_20820A1/include/hal/wiced_hal_nvram.h
```

```
/** Defines the valid range for vs_id's available to application. */
enum
{
    WICED_NVRAM_VSID_START = 0x200,
    WICED_NVRAM_VSID_END   = 0x3FFF,
};
```

See how it is test in “APPLICATION_START”:

```
// Check if NVRAM works or not
wiced_bt_device_link_keys_t nvram_keys;
if (app_read_nvram(SPP_NVRAM_ID, &nvram_keys, sizeof(wiced_bt_device_link_keys_t)) == 0)
{
    nvram_keys.bd_addr[0] = 0;
    nvram_keys.bd_addr[1] = 0;
    WICED_BT_TRACE("Cannot read NVRAM...\r\n");
    if (app_write_nvram(SPP_NVRAM_ID, sizeof(wiced_bt_device_link_keys_t), &nvram_keys) == 0)
    {
        WICED_BT_TRACE("Cannot write NVRAM...\r\n");
    }
}
```

# Send report (BLE HID)

```
hci_ble_hid_dev_handle_command - length = 15 - First = 2
Opcode:0602
HCI_CONTROL_HID_COMMAND_SEND_REPORT: 01 01 01 02 00 00 00 00 00 00 00
                                                                  send report
 type:1 id:0x1 length:9 conn_id:1
Sending notification... : attr_handle 50, length 8,
hci_ble_hid_dev_handle_command - length = 15 - First = 2
Opcode:0602
HCI_CONTROL_HID_COMMAND_SEND_REPORT: 01 01 01 00 00 00 00 00 00 00 00
                                                                  send report
 type:1 id:0x1 length:9 conn_id:1
Sending notification... : attr_handle 50, length 8,
```

Remark: you can use the ”ClientControl.exe” file to test some controls.

| | |
|---|---|
| | HCI_CONTROL_HID_COMMAND_SEND_REPORT: 01 01 01 00 00 1e 00 00 00 00 00 |
| 2 | HCI_CONTROL_HID_COMMAND_SEND_REPORT: 01 01 01 00 00 1f 00 00 00 00 00 |
| a | HCI_CONTROL_HID_COMMAND_SEND_REPORT: 01 01 01 00 00 04 00 00 00 00 00 |

**Table 12: Keyboard/Keypad Page**

| Usage ID (Dec) | Usage ID (Hex) | Usage Name | Ref: Typical AT-101 Position | PC-AT | Mac | UNIX | Boot |
|---|---|---|---|---|---|---|---|
| 0 | 00 | Reserved (no event indicated)[9] | N/A | √ | √ | √ | 4/101/104 |
| 1 | 01 | Keyboard ErrorRollOver[9] | N/A | √ | √ | √ | 4/101/104 |
| 2 | 02 | Keyboard POSTFail[9] | N/A | √ | √ | √ | 4/101/104 |
| 3 | 03 | Keyboard ErrorUndefined[9] | N/A | √ | √ | √ | 4/101/104 |
| 4 | 04 | Keyboard a and A[4] | 31 | √ | √ | √ | 4/101/104 |
| 5 | 05 | Keyboard b and B | 50 | √ | √ | √ | 4/101/104 |
| 6 | 06 | Keyboard c and C[4] | 48 | √ | √ | √ | 4/101/104 |
| 7 | 07 | Keyboard d and D | 33 | √ | √ | √ | 4/101/104 |
| 8 | 08 | Keyboard e and E | 19 | √ | √ | √ | 4/101/104 |
| 9 | 09 | Keyboard f and F | 34 | √ | √ | √ | 4/101/104 |

The usual format for keyboard reports is the following byte array:

```
[modifier, reserved, Key1, Key2, Key3, Key4, Key6, Key7]
```

Note that this report can send six simultaneous key events. Their order in the report doesn't matter, since the OS must keep track of the timing of each key press.

The reserved byte might be used by vendors, but is mostly useless.

When you press the letter 'a' on a USB keyboard, the following report will be sent over the USB interrupt pipe:

```
'a' report:     [0, 0, 4, 0, 0, 0, 0, 0]
```

```
'A' report:     [2, 0, 4, 0, 0, 0, 0, 0]
```
→ Modifier set to 2.

**Input report**: “keyboard” sends information.

**Output report**: Computer can control LEDs of the device

**Feature report**: Computer can control from internal properties of the device.

## Report descriptor

Report descriptor: device can describe what the report looks like (see file hci_ble_hid_dev_db.c – line 172).

```
0x05 , 0x01,                    // USAGE_PAGE (Generic Desktop)
0x09 , 0x06,                    // USAGE (Keyboard)
0xA1 , 0x01,                    // COLLECTION (Application)
0x85 , 0x01,                    //    REPORT_ID (1)
0x75 , 0x01,                    //    REPORT_SIZE (1)
```

See documents: https://www.usb.org/sites/default/files/documents/hid1_11.pdf and https://www.usb.org/sites/default/files/documents/hut1_12v2.pdf

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 0x05 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 |
| | Function of the item = 0<br>Usage page | | | | Type of item = 1<br>Global | | Length of data = 1 | |
| 0x01 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |
| | Generic Desktop Controls (HUT document – section 3 Usage Pages) | | | | | | | |
| 0x09 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1 |
| | Function of the item = 0<br>Usage | | | | Type of item = 2<br>Local | | Length of data = 1 | |
| 0x06 | We have "Generic Desktop Controls" so we have to look at section 4 (see section 3 Usage pages in HUT).<br>In section 4 (generic desktop page) we see: 0x06 -> Keyboard. | | | | | | | |
| 0xA1 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 1 |
| | Function of the item = 0xA = 10<br>Collection | | | | Type of item = 0<br>Main | | Length of data = 1 | |

```
0x05 , 0x01,                    // USAGE_PAGE (Generic Desktop)
0x09 , 0x06,                    // USAGE (Keyboard)
0xA1 , 0x01,                    // COLLECTION (Application)
0x85 , 0x01,                    //    REPORT_ID (1)
0x75 , 0x01,                    //    REPORT_SIZE (1)
0x95 , 0x08,                    //    REPORT_COUNT (8)
0x05 , 0x07,                    //    USAGE_PAGE (Keyboard)
```

Sending 8 bytes of information (for the report type 1).

```
0x05, 0x0C,                     // USAGE_PAGE (Consumer Devices)
0x09, 0x01,                     // USAGE (Consumer Control)
0xA1, 0x01,                     // COLLECTION (Application)
0x85, 0x02,                     //      Report ID (2)
0x75, 0x10,                     //      REPORT_SIZE (10)
0x95, 0x02,                     //      REPORT_COUNT (2)
```

Sending 2 bytes of information (for the report type 2).

# Connected or not

**Is BT connected?**

Check the value of “spp_handle”.

If not 0 -> connected.

**Is BLE HID connected?**

Check the value of “hci_ble_hid_dev_state.conn_id”.

If not 0 -> connected.

# Update battery level

```
static void ble_hid_battery_level(uint8_t new_level)
{
    uint16_t attr_handle = HDLC_BATTERY_SERVICE_BATTERY_LEVEL_VALUE;

    if ( hci_ble_hid_dev_state.paired && ( hci_ble_hid_dev_state.conn_id == 0 ) )
    {
        WICED_BT_TRACE("Paired, but no connection id...\r\n");
    }
    else
    {
        hci_ble_hid_dev_battery_service_battery_level[0] = new_level;
        wiced_bt_gatt_status_t gatt_status = wiced_bt_gatt_send_notification( hci_ble_hid_dev_state.conn_id, attr_handle, sizeof(uint8_t), hci_ble_hid_dev_battery_service_battery_level);
        if (gatt_status != WICED_BT_GATT_SUCCESS)
            WICED_BT_TRACE("Err: wiced_bt_gatt_send_notification failed %d\r\n", gatt_status);
    }
}
```

# Useful links

https://novelbits.io/bluetooth-hid-devices-an-intro/

https://github.com/microsoft/hidtools

https://github.com/Infineon/btsdk-docs/tree/master